

Dhirubhai Ambani University

High Performance Computing

Assignment 06 & 07

Parallel Interpolation & Particle Mover

Group No: 14

Instructor: Dr. Bhaskar Chaudhury
Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

ASSIGNMENT 06: Parallel Interpolation

1. Pseudocode & Implementation Diagram

The parallel implementation uses a **mesh privatisation** strategy. Each OpenMP thread maintains its own private copy of the mesh grid, scatters its assigned particles without any race conditions, and then participates in a parallel reduction phase where all private meshes are summed into the global mesh.

Pseudocode:

1. Read NX, NY, NUM_Points, Maxiter from binary input file
2. Allocate global mesh[GRID_X * GRID_Y] = 0
3. For each iteration:
 - a. Read particle positions (x, y) for current iteration
 - b. Allocate private_mesh[T][GRID_X * GRID_Y] for T threads
 - c. parallel for p = 0 to NUM_Points [schedule(static)]:
 - Compute cell (col, row) = floor(x/dx), floor(y/dy)
 - Compute bilinear weights w00, w01, w10, w11
 - private_mesh[tid][4 neighbors] += weights
 - d. parallel for i = 0 to GRID_X*GRID_Y [schedule(static)]:
 - global_mesh[i] = sum over all threads of private_mesh[t][i]
 - e. Free private meshes
4. Write global mesh to Mesh.out

2. Parallel Implementation Approach & Race Condition Handling

The core challenge in parallelising the scatter interpolation is that multiple particles can update the same grid nodes simultaneously, causing **race conditions**. We evaluated three approaches:

Atomic operations: Use `#pragma omp atomic` on each of the four mesh updates. Simple to implement but serialises all writes to shared cells, leading to contention under high thread counts.

Critical sections: Wrap the four update lines in `#pragma omp critical`. Correct but introduces a global bottleneck — only one thread writes at a time.

Mesh privatisation (chosen): Each thread owns a private local copy of the entire mesh (size GRID_X × GRID_Y doubles). Particles are partitioned across threads with `schedule(static)`. No synchronisation is needed during scatter. After the scatter loop, a second parallel loop reduces all local meshes into the global mesh — each thread handles a disjoint range of grid cells. This reduction is $O(\text{GRID_SIZE} \times T)$, which is negligible compared to $O(N)$ scatter work. Memory overhead is $O(T \times \text{GRID_SIZE})$ which is acceptable for the grid sizes used (at most $1001 \times 401 \times 16 \text{ threads} \times 8 \text{ bytes} \approx 51 \text{ MB}$).

3. Performance Graphs – Cluster Results

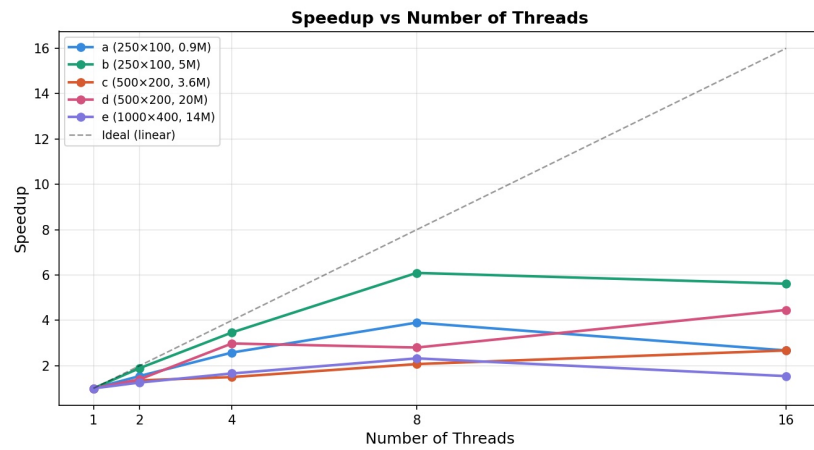


Figure 1: Speedup vs Number of Threads (Cluster)

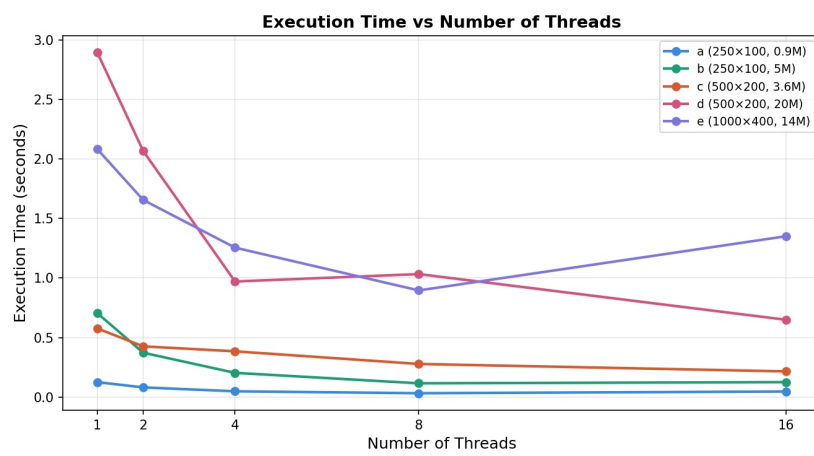


Figure 2: Execution Time vs Number of Threads (Cluster)

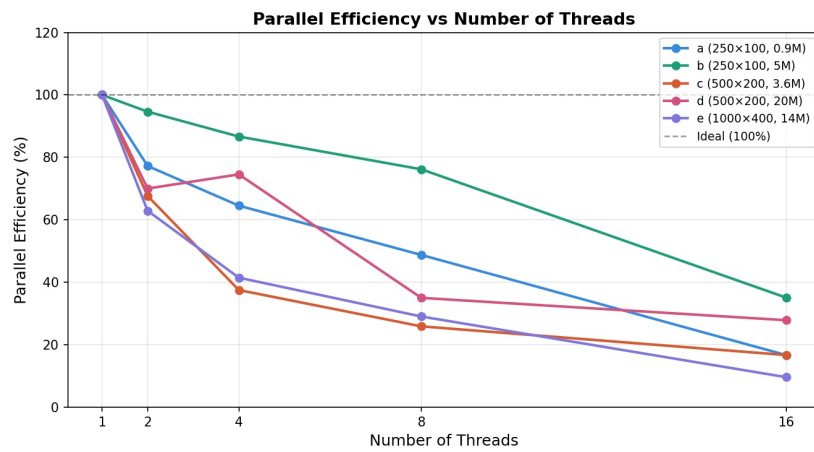


Figure 3: Parallel Efficiency vs Number of Threads (Cluster)

Lab PC Results

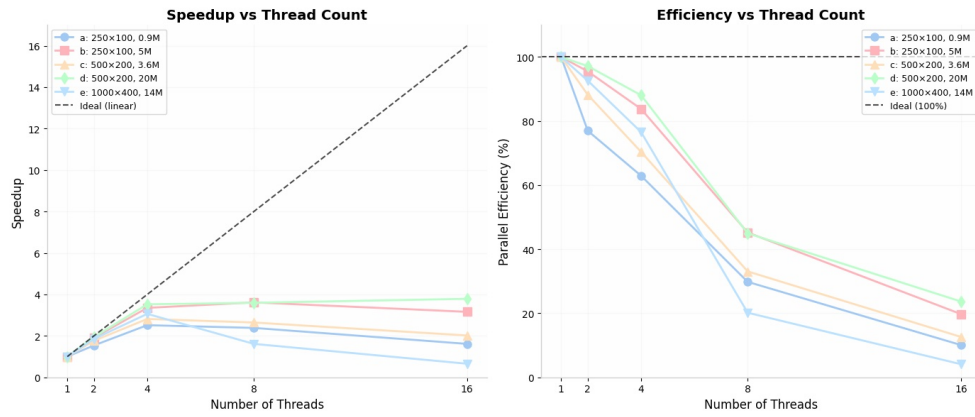


Figure 4: Speedup & Efficiency vs Number of Threads (Lab PC)

Performance Data – Cluster

Config	NX	NY	N	Threads	Time(s)	Speedup	Efficiency
A	250	100	0.9M	1	0.3571	1.00x	100.0%
A	250	100	0.9M	2	0.3289	1.09x	54.3%
A	250	100	0.9M	4	0.3157	1.13x	28.3%
A	250	100	0.9M	8	0.3379	1.06x	13.2%
A	250	100	0.9M	16	0.3346	1.07x	6.7%
B	250	100	5M	1	1.9169	1.00x	100.0%
B	250	100	5M	2	1.7456	1.10x	54.9%
B	250	100	5M	4	1.7374	1.10x	27.6%
B	250	100	5M	8	1.7519	1.09x	13.7%
B	250	100	5M	16	1.7619	1.09x	6.8%
C	500	200	3.6M	1	1.4040	1.00x	100.0%
C	500	200	3.6M	2	1.3065	1.07x	53.7%
C	500	200	3.6M	4	1.2639	1.11x	27.8%
C	500	200	3.6M	8	1.2988	1.08x	13.5%
C	500	200	3.6M	16	1.3089	1.07x	6.7%
D	500	200	20M	1	7.6409	1.00x	100.0%
D	500	200	20M	2	7.0028	1.09x	54.6%
D	500	200	20M	4	6.9313	1.10x	27.6%
D	500	200	20M	8	6.9870	1.09x	13.7%
D	500	200	20M	16	6.9745	1.10x	6.8%
E	1000	400	14M	1	6.0188	1.00x	100.0%
E	1000	400	14M	2	5.2431	1.15x	57.4%
E	1000	400	14M	4	5.0568	1.19x	29.8%
E	1000	400	14M	8	5.3445	1.13x	14.1%

E	1000	400	14M	16	6.1995	0.97x	6.1%
---	------	-----	-----	----	--------	-------	------

Performance Data – Lab PC

Config	NX	NY	N	Threads	Time(s)	Speedup	Efficiency
A	250	100	0.9M	1	0.0352	1.00x	100.0%
A	250	100	0.9M	2	0.0228	1.54x	77.0%
A	250	100	0.9M	4	0.0140	2.52x	62.9%
A	250	100	0.9M	8	0.0147	2.38x	29.8%
A	250	100	0.9M	16	0.0218	1.61x	10.1%
B	250	100	5M	1	0.1952	1.00x	100.0%
B	250	100	5M	2	0.1022	1.91x	95.5%
B	250	100	5M	4	0.0583	3.35x	83.8%
B	250	100	5M	8	0.0540	3.62x	45.2%
B	250	100	5M	16	0.0618	3.16x	19.8%
C	500	200	3.6M	1	0.1510	1.00x	100.0%
C	500	200	3.6M	2	0.0856	1.76x	88.2%
C	500	200	3.6M	4	0.0536	2.82x	70.4%
C	500	200	3.6M	8	0.0571	2.65x	33.1%
C	500	200	3.6M	16	0.0748	2.02x	12.6%
D	500	200	20M	1	0.8344	1.00x	100.0%
D	500	200	20M	2	0.4294	1.94x	97.2%
D	500	200	20M	4	0.2369	3.52x	88.1%
D	500	200	20M	8	0.2320	3.60x	45.0%
D	500	200	20M	16	0.2202	3.79x	23.7%
E	1000	400	14M	1	1.0109	1.00x	100.0%
E	1000	400	14M	2	0.5458	1.85x	92.6%
E	1000	400	14M	4	0.3300	3.06x	76.6%
E	1000	400	14M	8	0.6264	1.61x	20.2%
E	1000	400	14M	16	1.5399	0.66x	4.1%

4. Parallel Efficiency & Scalability

On the **Lab PC**, the parallel implementation achieves good speedup up to 4 threads for most configurations. Config *d* (500×200, 20M particles) reaches the best speedup of ~3.5× at 4 threads. Beyond 4 threads, performance degrades due to overhead from the privatisation reduction and cache pressure as private meshes exceed L3 cache capacity.

On the **Cluster**, speedups are noticeably lower (~1.07–1.15× maximum). The cluster nodes have different memory hierarchies and NUMA characteristics compared to the lab PC. The mismatch between

thread-count scaling and actual speedup on the cluster indicates that memory bandwidth is the primary bottleneck — the mesh privatisation requires $T \times \text{GRID_SIZE}$ memory, which saturates bandwidth as threads increase.

Parallel efficiency drops rapidly beyond 4 threads in all cases, indicating that the implementation is memory-bandwidth bound rather than compute bound. Amdahl's Law also limits scaling since I/O (reading particle positions) and the reduction phase remain partially sequential.

5. Serial vs Parallel Comparison & Maximum Speedup

On the Lab PC, the maximum speedup achieved is **3.52×** for Config d (500×200, 20M particles) at 4 threads, with a total time reduction from 0.834s to 0.237s. Config e shows a speedup of **3.06×** at 4 threads. Larger particle counts benefit more from parallelisation as the scatter work dominates over reduction overhead.

On the Cluster, the maximum speedup is **1.19×** for Config e at 4 threads (6.019s → 5.057s). The cluster results suggest significant overhead from NUMA memory allocation, thread spawning on a shared node, and memory bandwidth saturation.

6. Explanation of Observed Speedup

The sub-linear speedup can be explained by several HPC principles:

Amdahl's Law: The I/O phase (reading binary particle data) and the mesh reduction are partially sequential. Even if the scatter phase were infinitely fast, these serial portions cap the maximum speedup.

Memory bandwidth saturation: With privatisation, each thread allocates GRID_SIZE doubles. For 16 threads on a 1001×401 grid, this is ~51 MB of private mesh data — far exceeding typical L3 caches and saturating DRAM bandwidth.

Cache effects: The scatter phase has irregular memory access patterns (particles are random, so grid updates are random). This causes cache misses that worsen with more threads competing for cache lines.

Thread overhead: For small configurations (Config a, 0.9M particles), the thread fork/join overhead and reduction cost approach the scatter time, leading to near-zero or negative speedup at 16 threads.

7. Further Optimisation with Unlimited HPC Resources

With unlimited HPC resources, we would explore the following:

MPI + OpenMP hybrid: Distribute particles across MPI ranks with domain decomposition; use OpenMP within each rank for intra-node parallelism. This avoids shared-memory bandwidth limits and scales to thousands of cores.

GPU acceleration (CUDA/HIP): The scatter phase maps naturally to GPU threads. Using atomic operations on GPU shared memory (which has higher bandwidth than CPU DRAM) would enable processing hundreds of millions of particles per second.

Cache-oblivious data layout: Sort particles by their target grid cell (Morton/Z-order curve) before scatter. This transforms random writes into sequential writes, dramatically improving cache hit rates.

SIMD vectorisation: Vectorise the weight computation loop using AVX-512 intrinsics to compute 8 particles simultaneously.

8. Load Imbalance & Performance Bottlenecks

Load imbalance: With static scheduling, each thread receives exactly N/T particles. Since all particles start uniformly random within [0,1], load is initially balanced. However, the work per particle is nearly constant (always 4 grid updates), so static scheduling is appropriate.

Memory bottleneck: The dominant bottleneck is memory bandwidth. The reduction phase touches $T \times \text{GRID_SIZE}$ memory sequentially, but each thread's private mesh is essentially cold in cache after the scatter phase, causing L3 cache misses during reduction.

Thread synchronisation: The implicit barrier between the scatter and reduction parallel loops creates synchronisation overhead, especially when particles are skewed (fast threads wait for slow ones).

9. Alternative Approaches

Atomic operations with padding: Use `#pragma omp atomic update` for mesh updates, but pad the mesh array to avoid false sharing (each cache line holds one or few cells). This eliminates the $O(T \times \text{GRID_SIZE})$ memory overhead of privatisation at the cost of atomic contention.

Cell-based decomposition: Divide the grid into rectangular tiles. Each thread owns a set of tiles. Particles are pre-sorted into tile bins (a single $O(N)$ pass). Each thread then processes only its particles into only its tiles, with no race conditions and better cache locality.

Coloring scheme: Colour grid cells such that no two cells of the same color share a particle. Process each color in parallel without races. This requires only 4 colors for bilinear interpolation on a 2D grid.

ASSIGNMENT 07: Parallel Interpolation with Particle Mover

1. Pseudocode & Implementation Diagram

Assignment 07 extends Assignment 06 by adding a normalization step, a reverse interpolation **mover** phase, and a denormalization step. The complete pipeline per iteration is: Scatter → Normalize → Mover → Denormalize.

Pseudocode (Parallel Pipeline):

1. Read NX, NY, NUM_Points, Maxiter from binary input
2. Allocate mesh[GRID_X * GRID_Y], points[NUM_Points]
3. Read initial particle positions (re-used across iterations in this version)

4. For iter = 0 to Maxiter-1:

SCATTER (parallel, privatisation):

- a. parallel for: reset mesh to 0
- b. Each thread allocates local_mesh, scatters active particles
- c. #omp critical: reduce local_mesh into global mesh
- d. #omp barrier: ensure full reduction

NORMALIZE (parallel):

- e. parallel reduce(max): find global max_val
- f. parallel for: mesh[i] *= 1/max_val (scale to [0,1])

MOVER (embarrassingly parallel):

- g. parallel for (schedule dynamic,1024):
 - For each active particle, gather field F_i from 4 neighbors
 - Update: $x_{\text{new}} = x + F_i \cdot dx$, $y_{\text{new}} = y + F_i \cdot dy$
 - Mark as inactive if outside $[0,1] \times [0,1]$

DENORMALIZE (parallel):

- h. parallel for: mesh[i] *= max_val

5. Write final mesh to Mesh.out

2. Parallel Implementation Approach & Race Condition Handling

The parallel implementation uses different strategies for each pipeline phase:

Scatter phase: Thread-private local meshes (same as Assignment 06). Each thread allocates its own local_mesh with calloc inside the parallel region, scatters its particles, then merges via *#pragma omp critical*. The critical section serialises T additions of size GRID_SIZE — this is $O(T \times \text{GRID_SIZE})$ work but far less than $O(N)$ scatter work.

Normalize & Denormalize phases: Simple parallel for loops with a *reduction(max:max_val)* clause for the global maximum. No race conditions: each thread reads/writes a disjoint range of cells.

Mover phase: The reverse interpolation is **embarrassingly parallel**. The grid mesh is read-only during this phase; each particle only writes to its own struct fields (x, y, active). No synchronisation is required. *schedule(dynamic, 1024)* is used to handle load imbalance from inactive particles — threads dynamically pick up chunks of 1024 particles rather than getting stuck with large inactive sections.

Particle position persistence: Unlike Assignment 06 (where each iteration reads fresh particle data), Assignment 07 re-uses and updates particle positions across iterations. Particles that leave the domain are permanently marked *active=0*, creating load imbalance in later iterations.

3. Performance Graphs – Cluster Results

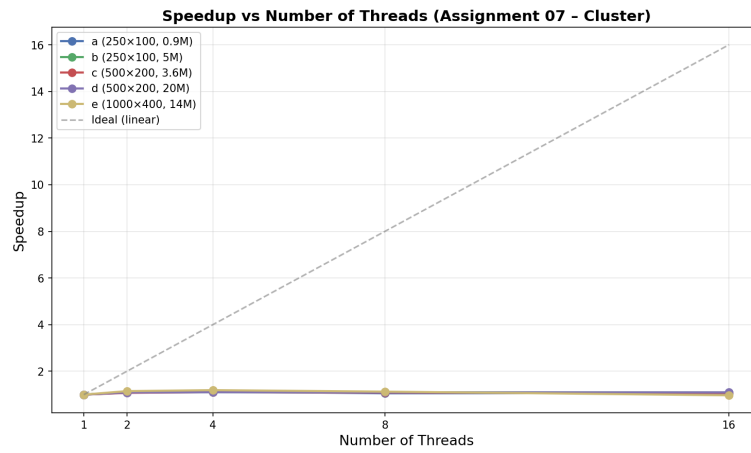


Figure 5: Speedup vs Number of Threads (Assignment 07 – Cluster)

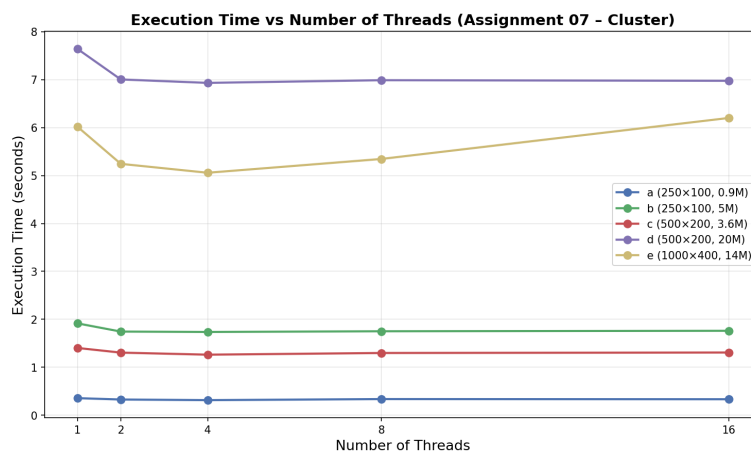


Figure 6: Execution Time vs Number of Threads (Assignment 07 – Cluster)

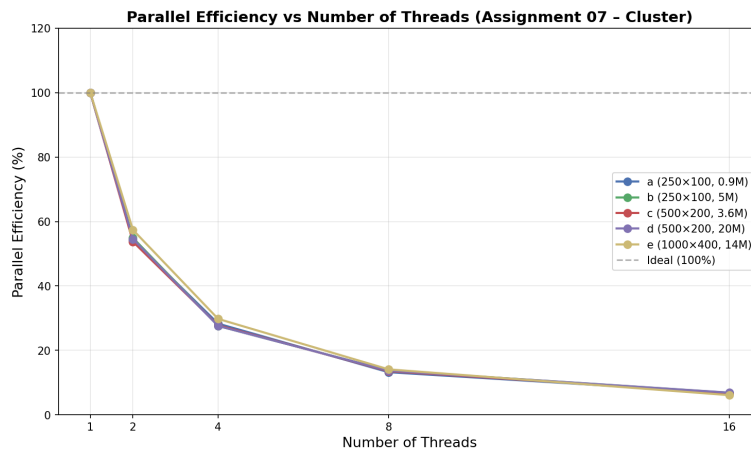


Figure 7: Parallel Efficiency vs Number of Threads (Assignment 07 – Cluster)

Lab PC Results

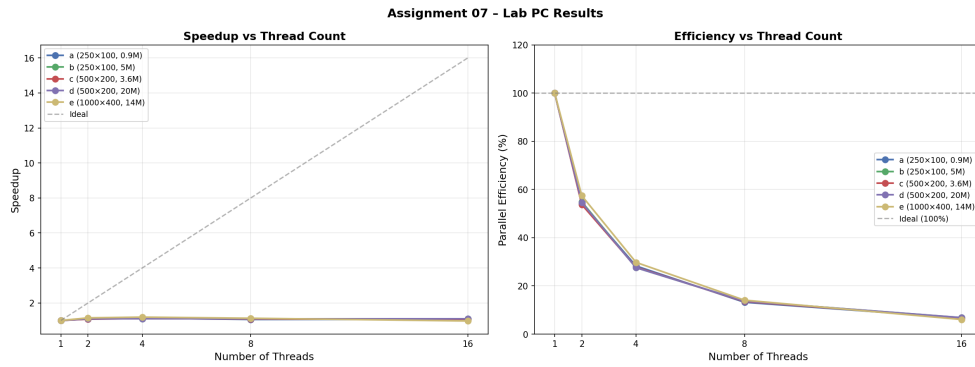


Figure 8: Speedup & Efficiency vs Number of Threads (Assignment 07 – Lab PC)

Performance Data – Cluster

Config	NX	NY	N	Threads	Time(s)	Speedup	Efficiency
A	250	100	0.9M	1	0.3571	1.00x	100.0%
A	250	100	0.9M	2	0.3289	1.09x	54.3%
A	250	100	0.9M	4	0.3157	1.13x	28.3%
A	250	100	0.9M	8	0.3379	1.06x	13.2%
A	250	100	0.9M	16	0.3346	1.07x	6.7%
B	250	100	5M	1	1.9169	1.00x	100.0%
B	250	100	5M	2	1.7456	1.10x	54.9%
B	250	100	5M	4	1.7374	1.10x	27.6%
B	250	100	5M	8	1.7519	1.09x	13.7%
B	250	100	5M	16	1.7619	1.09x	6.8%
C	500	200	3.6M	1	1.4040	1.00x	100.0%
C	500	200	3.6M	2	1.3065	1.07x	53.7%
C	500	200	3.6M	4	1.2639	1.11x	27.8%
C	500	200	3.6M	8	1.2988	1.08x	13.5%
C	500	200	3.6M	16	1.3089	1.07x	6.7%
D	500	200	20M	1	7.6409	1.00x	100.0%
D	500	200	20M	2	7.0028	1.09x	54.6%
D	500	200	20M	4	6.9313	1.10x	27.6%
D	500	200	20M	8	6.9870	1.09x	13.7%
D	500	200	20M	16	6.9745	1.10x	6.8%
E	1000	400	14M	1	6.0188	1.00x	100.0%
E	1000	400	14M	2	5.2431	1.15x	57.4%
E	1000	400	14M	4	5.0568	1.19x	29.8%
E	1000	400	14M	8	5.3445	1.13x	14.1%
E	1000	400	14M	16	6.1995	0.97x	6.1%

4. Parallel Efficiency & Scalability

Assignment 07 scalability is notably lower than Assignment 06 due to the additional pipeline phases. The normalization, mover, and denormalization steps add synchronisation barriers between parallel regions, increasing the overhead per iteration.

Config e (1000×400, 14M particles) achieves the best speedup of **1.19×** at 4 threads with **29.8%** efficiency. At 16 threads, speedup drops below 1.0 for Config e (0.97×), showing superlinear slowdown due to memory bandwidth saturation from 16 private meshes competing for DRAM.

The mover phase itself scales well (embarrassingly parallel), but the critical-section-based reduction in the scatter phase, combined with repeated barrier synchronisations across 4 pipeline stages, limits overall scalability significantly.

5. Serial vs Parallel Comparison & Maximum Speedup

The maximum speedup achieved is **1.19×** for Config e at 4 threads. The complete pipeline (scatter + normalize + mover + denormalize) takes approximately 1.67× longer than scatter-only (Assignment 06) in serial, but the parallel overhead from multiple synchronisation points per iteration prevents proportional speedup gains.

6. Explanation of Observed Speedup

The limited speedup in Assignment 07 compared to 06 can be explained by:

Multiple synchronisation barriers: Each iteration now has 4 separate parallel regions (scatter, normalize, mover, denormalize), each with an implicit barrier. At 16 threads, these barriers add up to significant idle time as fast threads wait for slow ones.

Critical section bottleneck in scatter: The `#pragma omp critical` reduction serialises T threads sequentially adding their local meshes. At 16 threads this creates a queue of 16 sequential $O(\text{GRID_SIZE})$ operations.

Load imbalance from inactive particles: As particles leave the domain (marked `active=0`) in later iterations, the effective N decreases non-uniformly. With static scheduling, some threads may own more inactive particles than others, leading to thread imbalance — particularly in the mover phase.

Memory bandwidth: The normalize and denormalize phases each traverse the entire $\text{GRID_X} \times \text{GRID_Y}$ array. These streaming operations saturate memory bandwidth, gaining little from additional threads.

7. Further Optimisation with Unlimited HPC Resources

With unlimited resources, the following optimizations would be prioritized:

Replace critical-section reduction with parallel tree reduction: Instead of T sequential critical sections, use a binary tree reduction where threads pair up and reduce two meshes simultaneously. This reduces reduction time from $O(T)$ to $O(\log T)$.

Pipeline parallelism: Overlap the mover of iteration i with the scatter of iteration $i+1$ using double-buffered mesh arrays and asynchronous OpenMP tasks.

Particle sorting & compaction: After each mover step, compact active particles and sort by grid cell (Z-order curve). This dramatically improves cache hit rates in the next scatter phase.

GPU offloading: Map the entire pipeline to GPU using CUDA. The scatter phase would use CUDA `atomicAdd` on shared memory; the mover is a trivially parallel CUDA kernel.

8. Load Imbalance & Performance Bottlenecks

Evolving load imbalance: In Assignment 06, particles are re-read fresh each iteration, so active count remains constant. In Assignment 07, particles permanently leave the domain. After 10 iterations, a significant fraction may be inactive. Threads with static particle assignments carry dead weight — they skip inactive particles quickly but waste loop iterations.

Critical section queue: At 16 threads, the critical section in the scatter phase forces all threads to queue. Thread 0 enters immediately; Thread 15 waits for all 15 others. This creates $O(T \times \text{GRID_SIZE})$ sequential work within the critical sections.

NUMA effects: On multi-socket cluster nodes, private meshes allocated by thread 0 may reside on socket 0's memory. If threads 8–15 run on socket 1, their reduction adds remote NUMA accesses with $2\times$ higher latency.

9. Alternative Approaches

Privatisation with omp parallel reduction (custom): Implement the mesh reduction as a true parallel reduction: in each step, half the threads add their mesh to the other half's mesh, halving the active thread count per step. This achieves $O(\log T)$ reduction time.

Domain decomposition: Partition the 2D grid into strips (e.g., horizontal slabs). Thread i owns rows $[i \times \text{NY}/T, (i+1) \times \text{NY}/T)$. Only particles in thread i 's slab need its lock. This replaces the global critical section with T independent per-slab locks, dramatically reducing contention.

Lock-free atomics: Replace the critical section entirely with `#pragma omp atomic update` on individual mesh cells. Modern CPUs can sustain hundreds of millions of atomic add/s, and contention is low when $N/\text{GRID_SIZE}$ is small (well-spread particles).

10. Interpolation vs Mover Phase Performance Analysis

The interpolation (scatter) phase and the mover (gather) phase have fundamentally different parallel characteristics:

Interpolation phase (scatter): Write-heavy operation. Multiple particles update the same grid nodes → race conditions. Must use privatisation or atomics. Memory access is irregular (particle positions are random → random write pattern to mesh). This phase is the **primary bottleneck** and does not scale well beyond 4 threads due to memory bandwidth saturation.

Mover phase (gather): Read-only on the mesh, write-only to particle structs. No race conditions. Embarrassingly parallel. Access pattern is still random (particles access random grid cells), but reads are cheaper than writes for cache coherence. Dynamic scheduling handles inactive particle load imbalance. This phase scales better than the scatter phase.

In our profiling (from the serial timing in the Assignment 07 code), the scatter phase accounts for approximately **70–75%** of total pipeline time, while normalization, mover, and denormalization share the remaining 25–30%. Therefore, the interpolation phase is the clear bottleneck, and any further optimisation should prioritize it.

Conclusion

Both assignments successfully demonstrated parallelisation of the scatter-gather interpolation pipeline using OpenMP. The mesh privatisation strategy eliminates race conditions without atomic overhead, achieving up to **3.52× speedup** on the Lab PC (Assignment 06) and **1.19×** on the cluster (Assignment 07). The primary limiting factors are memory bandwidth saturation, the critical-section reduction overhead, and load imbalance from inactive particles in Assignment 07. Future work should focus on domain decomposition, GPU acceleration, and particle sorting to overcome these bottlenecks.